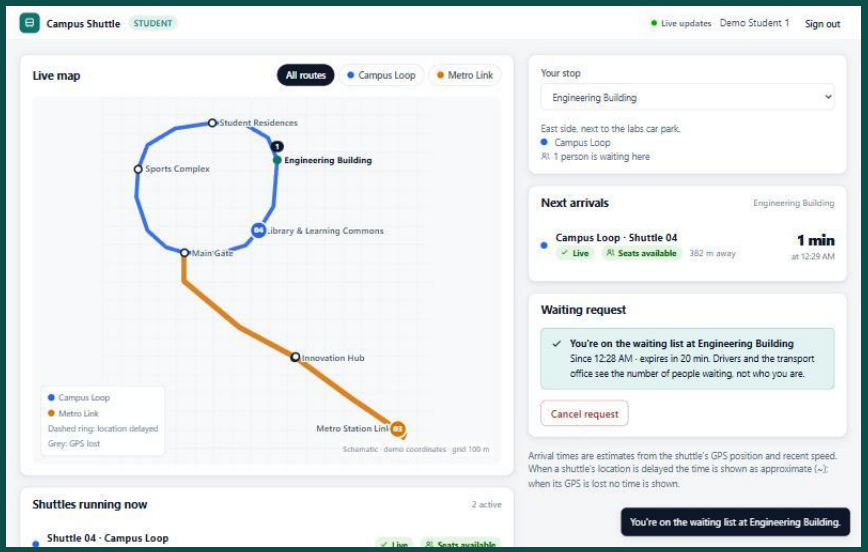
Campus Shuttle Tracking & Demand Prediction

Timely, honest shuttle information — and the demand data to plan with



Abdullah · Zuhair Halawa

BCS 305 Software Architecture · Dr. Khaled Ammar · Fall 2026

Students wait blind; the office plans without data

1 Students

Cannot see where the shuttle is, so they wait outdoors — often in the heat — or miss it.

We give: live map, arrival times with a confidence level, "I'm waiting here".

2 Drivers

Interrupted by calls asking where they are; blamed when GPS or mobile data fails.

We give: one-tap trips, offline-tolerant location sharing, a GPS simulator.

3 Transport office

Sees neither the fleet nor where and when students actually wait.

We give: live operations, routes, trip tracks, demand trends and explainable forecasts.

Also affected University IT, who must run it securely and cheaply · the data-protection officer, because driver locations and student identities are personal data · university management, who fund the fleet.

The hard quality: information that is timely and honest

When GPS and mobile data are good, every viewer must see a new position within about a second.

When they fail, the system must say so — "location delayed", "GPS lost" — instead of showing a confident but wrong arrival time, and recover — delivering positions buffered during a network outage, within stated limits.

Second hard concern: privacy of location and identity data.

35 ms

p95 to 1,000 simulated viewers (local benchmark, one PC)

20 / 60 s

no position → "delayed" → "GPS lost", no ETA

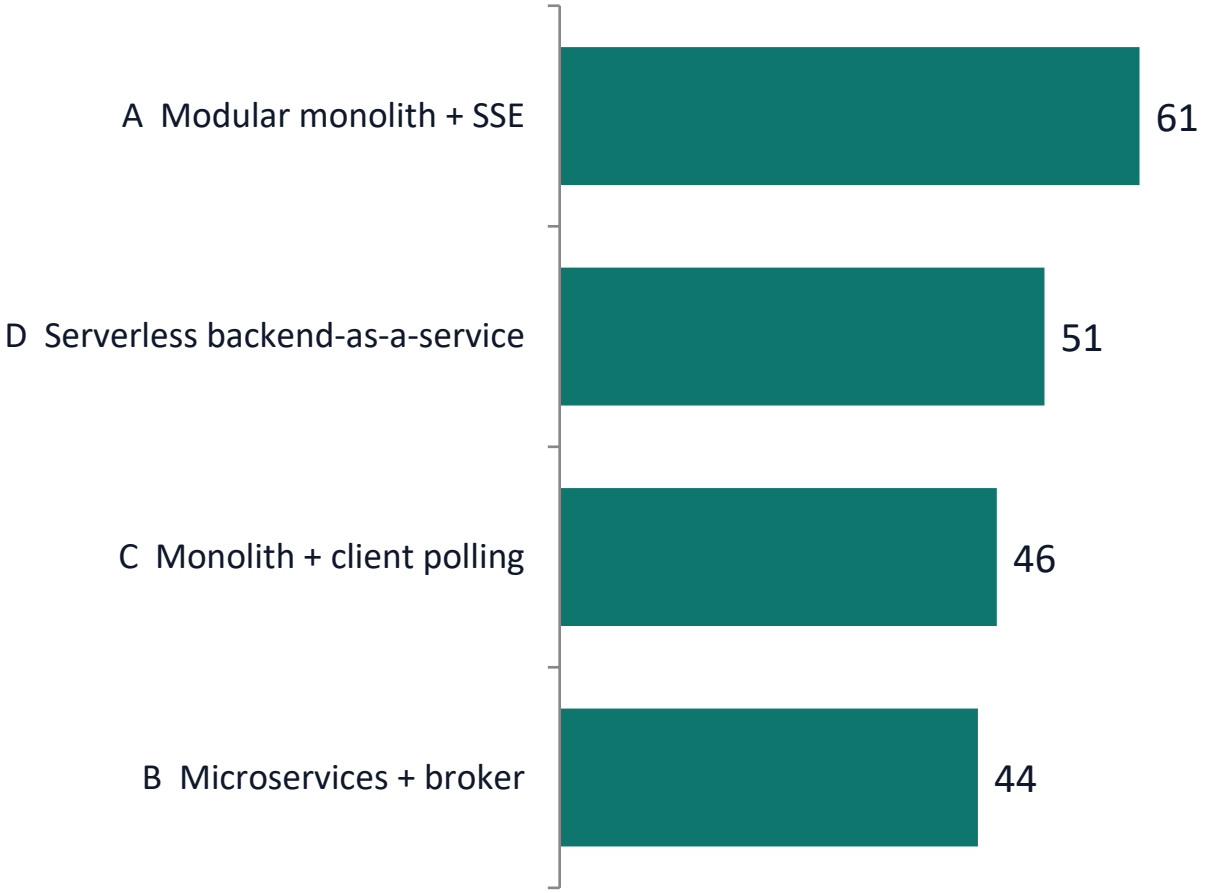
10 min

network outage with no validated position lost (page open; tested)

9

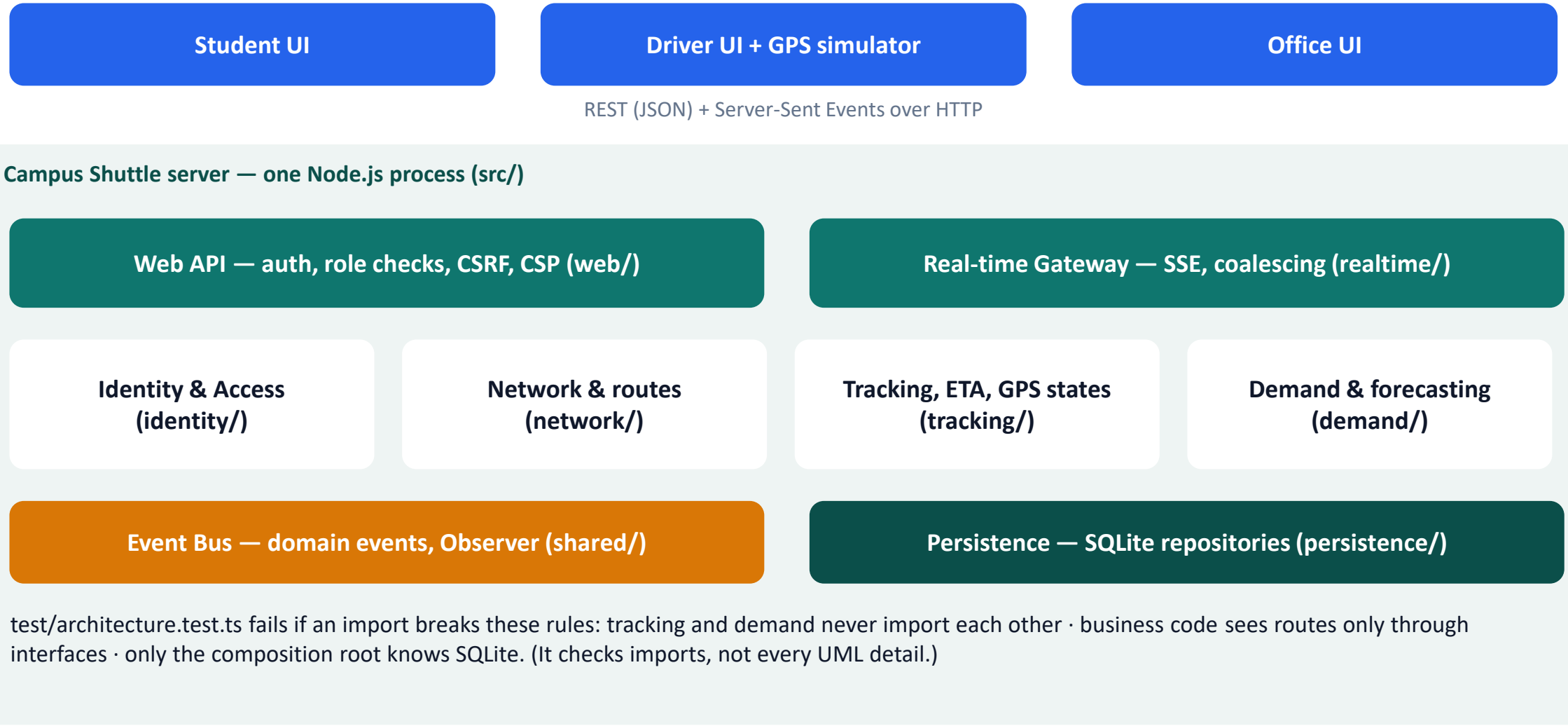
measurable quality-attribute scenarios (report §2.3)

Four architectures compared — the modular monolith wins



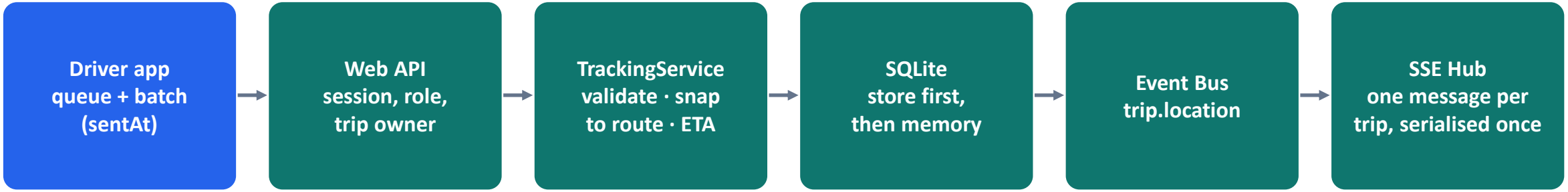
- Weighted against freshness, robustness, security & privacy, modifiability, team-of-two simplicity, growth
- A: best measured freshness, one trust boundary, one command to run
- B: independent scaling one campus does not need; ≥ 5 deployables
- C: data 2.5 s old on average; 200 req/s for 1,000 idle viewers
- D: location data held by a third party; paid tiers; hard offline demo

One process, eight components, enforced boundaries



test/architecture.test.ts fails if an import breaks these rules: tracking and demand never import each other · business code sees routes only through interfaces · only the composition root knows SQLite. (It checks imports, not every UML detail.)

One GPS upload reaches every viewer in milliseconds

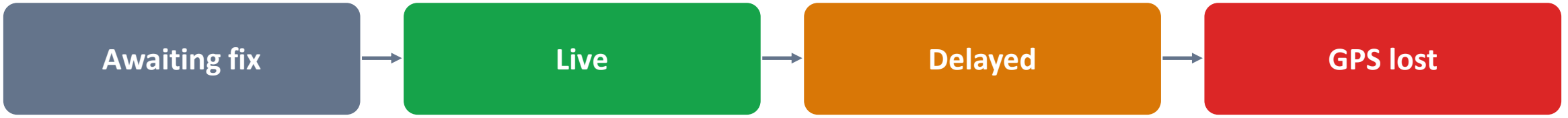


Local benchmark (1 PC, Node 24)	p50	p95	max	skipped
1,000 viewers × 60 updates	23–25 ms	35–37 ms	55–56 ms	0
3,000 viewers × 30 updates	70–72 ms	90–98 ms	108–113 ms	0

npm run load-test · viewers share the server process over loopback, one session, no browsers: a fan-out benchmark with simulated viewers, not a field test (31 Oct and 1 Nov runs)

- Server-Sent Events: browsers reconnect by themselves and get a full snapshot
- A slow phone skips updates and is resynchronised — it cannot grow server memory
- Positions are stored before memory changes: a restart loses nothing

When GPS fails, the system says so instead of guessing



Awaiting fix	Live	Delayed	GPS lost
trip started, no position yet	newest position ≤ 20 s old	20–60 s without a position	more than 60 s
Students see "waiting for first GPS position"	**Students see** ETA: "4 min · at 08:14"	**Students see** ETA marked "~4 min", then "Due"	**Students see** no ETA — "last seen 2 min ago"

Any accepted position → Live again · no position for 30 min → trip closed automatically

Network loss on the driver's phone

Positions are queued in the open driver page (newest 500 kept) and retried with backoff (1 → 15 s). On reconnection the server corrects the phone's clock using sentAt, sorts them, rejects duplicates, positions over 10 min old and impossible jumps, and the shuttle jumps to its true position. Closing the page loses the queue.

Five design patterns, each solving a concrete problem

1 Strategy

ETA and forecasting algorithms change without touching the services; models compared side by side.

EtaStrategy · DemandPredictor

2 Observer

One update reaches hundreds of browsers and the waiting module; tracking knows none of them.

EventBus · callback subscribers

3 State

What passengers may be told depends on GPS freshness; no status switch statements.

SignalState: Live / Stale / Lost

4 Repository

Business rules independent of SQL; unit tests run without a database.

TripRepository → SQLite / in-memory

5 Adapter

Demo without vehicles, but through exactly the same upload path as a real phone.

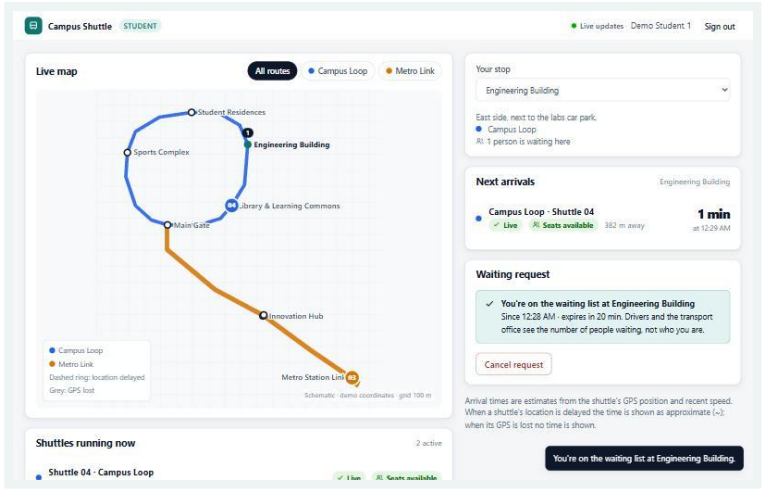
DeviceGps · SimulatedGps (duck-typed)

6 Composition root

One place knows concrete classes; tests swap the clock, database and strategies.

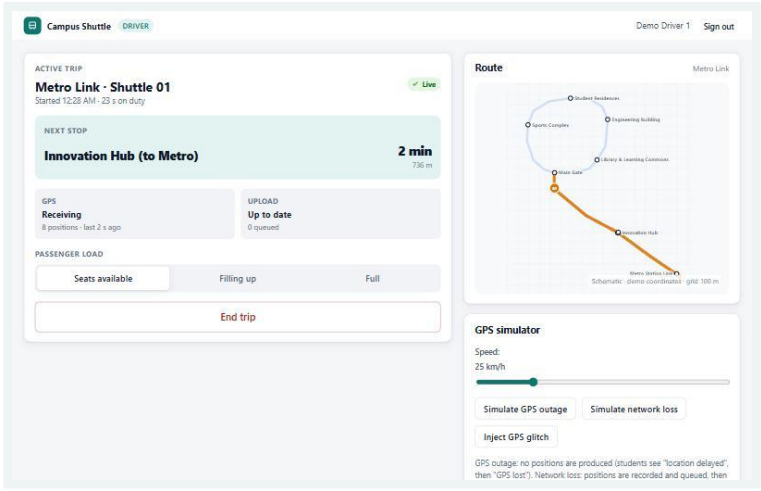
src/container.ts

Three roles, one live system



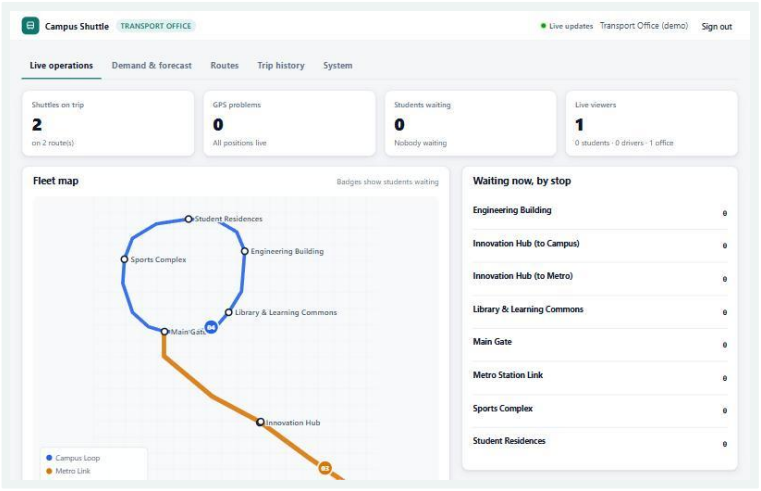
Student

Live map, arrivals with confidence, "I'm waiting here"



Driver

Trip status, what passengers see, GPS simulator



Transport office

Fleet, waiting demand, trips, routes, forecasts

Two primary use cases run end to end: run a trip → track it live, and waiting request → office demand view → forecast.

Demand forecasts the office can explain

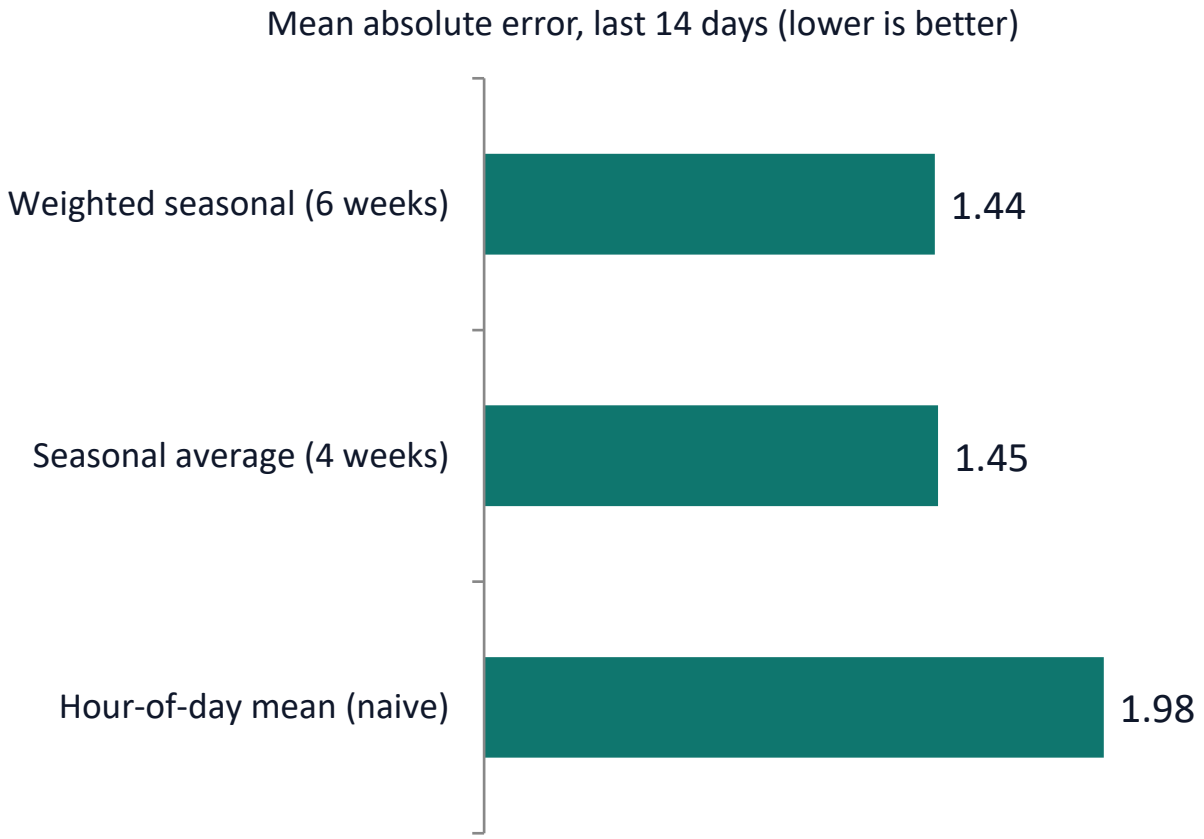


Each forecast shows the exact past values, weights and range it came from — and "no forecast" instead of a guessed zero.

ML lifecycle (RBC Borealis field guide)

1. **Discovery** Question: requests per stop and hour
2. **Feasibility** No real data → no trained model yet
3. **Design** Explanations and ranges for staff
4. **Development** Naive → seasonal → weighted baselines
5. **Production** Live requests become history automatically
6. **Maintenance** Backtest = monitoring; retrain only if it wins

Synthetic backtest: the method works, accuracy unproven



SYNTHETIC DATA

- History generated from our own written assumptions (weekday peaks, quiet weekends) — not from any university
- Weekday-aware models beat the naive one because the data was built that way: this checks the pipeline, nothing more
- With one semester of real requests, the same backtest decides whether a trained model is worth adding

Live demonstration

1 Run a trip and track it

- Driver starts a trip with simulated GPS
- Student sees the shuttle move and the ETA update
- Network loss → trip cannot end offline → queue delivered
- Injected GPS glitch is rejected
- GPS outage button → "delayed" (~20 s) → "GPS lost" (~60 s) → Live

2 Waiting request → demand → forecast

- Student taps "I'm waiting here"
- Office map badge and KPI update live
- Shuttle reaches the stop → request closed
- Forecast cell → explanation of its basis
- Office ends a trip with a reason → audit log

Set-up before the session

npm start · npm run simulate -- --outage --outage-at 30 · three private browser windows: student1, driver1, office1

Backup if anything fails live: docs/evidence/ (test run, load tests, evaluation)

Tested, measured — and honest about the gaps

103

automated tests, all passing (unit, integration, architecture)

26 × 4

endpoints × caller types checked for role access

113 ms

worst of 3,000 simulated viewers (local benchmark)

6

code defects found by tests and review, all fixed

Found by tests and independent review

- Shuttle snapped a few metres behind at route corners (load test)
- Real-time hub ran work after shutdown (integration test)
- Driver app gave up a trip the server had not ended; live streams outlived their session (review)

Not done yet

- Field test with a real phone on a real shuttle
- HTTPS, single sign-on, Arabic, screen-reader audit
- Real demand data — forecasts shown on synthetic data only
- UML diagrams rendered and checked visually; deployment load test

What we would do next

Field test

Real phones on a real route; tune the 20 s / 60 s thresholds and the speed blend

Production basics

HTTPS behind a reverse proxy, university sign-on, backups

Real data, then ML

One semester of requests → rerun the backtest → add a model only if it wins

Questions?

The prototype baseline was built with AI assistance (Claude Code); what we reviewed, changed and added is stated in our AI-use statement.